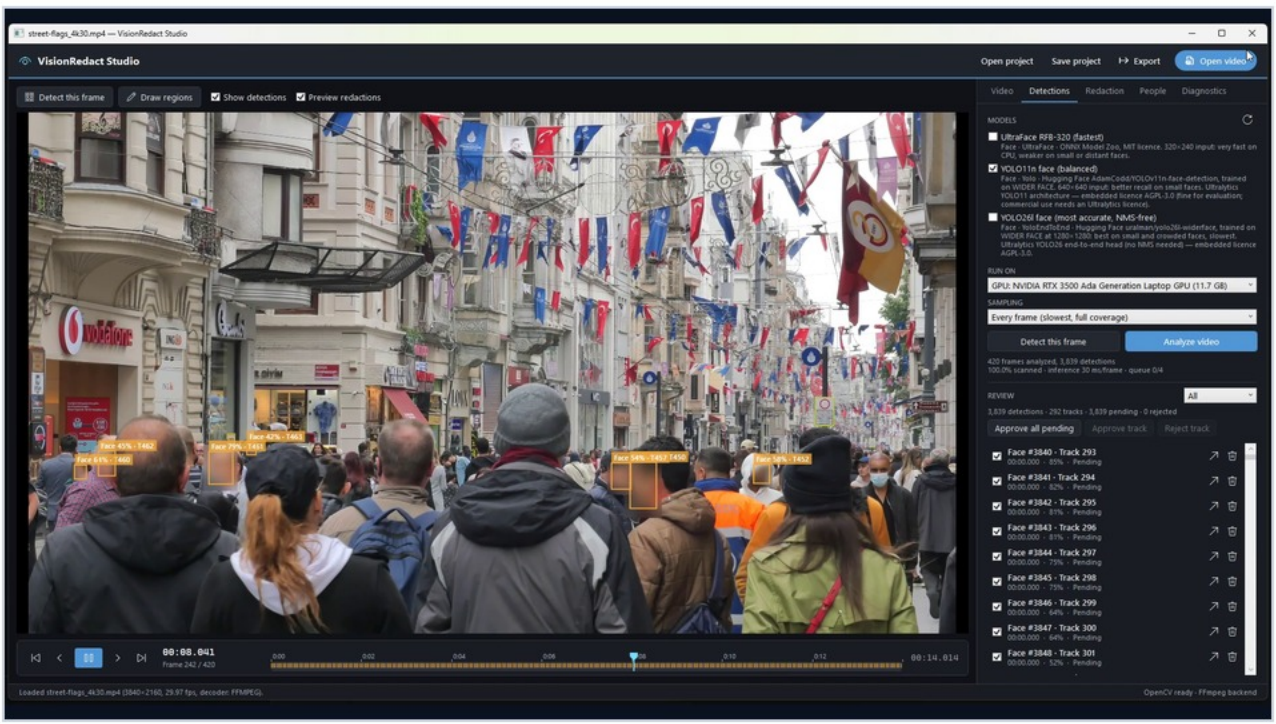
Local AI on the GPU in C#

An AI model on my laptop's GPU, inside a C# app, finds the faces in this 4K street, and not a single frame leaves the PC. How I plugged the AI in, made it fast, and turned faces into people. (It still misses some small or turned-away faces: that's Episode 4.)



C# .NET 10 YOLO Hugging Face ONNX Runtime DirectML

Samit Desai · linkedin.com/in/samit-desai-62868420

Three models, one frame

There's no single right answer. On this 4K street frame:



UltraFace — **2 faces · 3.1 ms a frame**
small, fast



YOLO11 nano — **6 faces · 5.4 ms a frame**
balanced



YOLO26 large — **22 faces · 74.0 ms a frame**
about 1 GB of graphics memory

More boxes isn't automatically more correct, and I'm not grading the models: the app lets you pick the trade-off today and plug in a faster model tomorrow.

Faces: on this one 4K frame. Times: the mean per frame on the RTX laptop GPU, over 420 frames of the same street at 1080p.

Inside the C# app, on any GPU

A model from Hugging Face

an .onnx file, nothing to install

↓

ONNX Runtime

in the app's own process: no Python, no server

↓

DirectML

any DirectX 12 GPU, not just NVIDIA

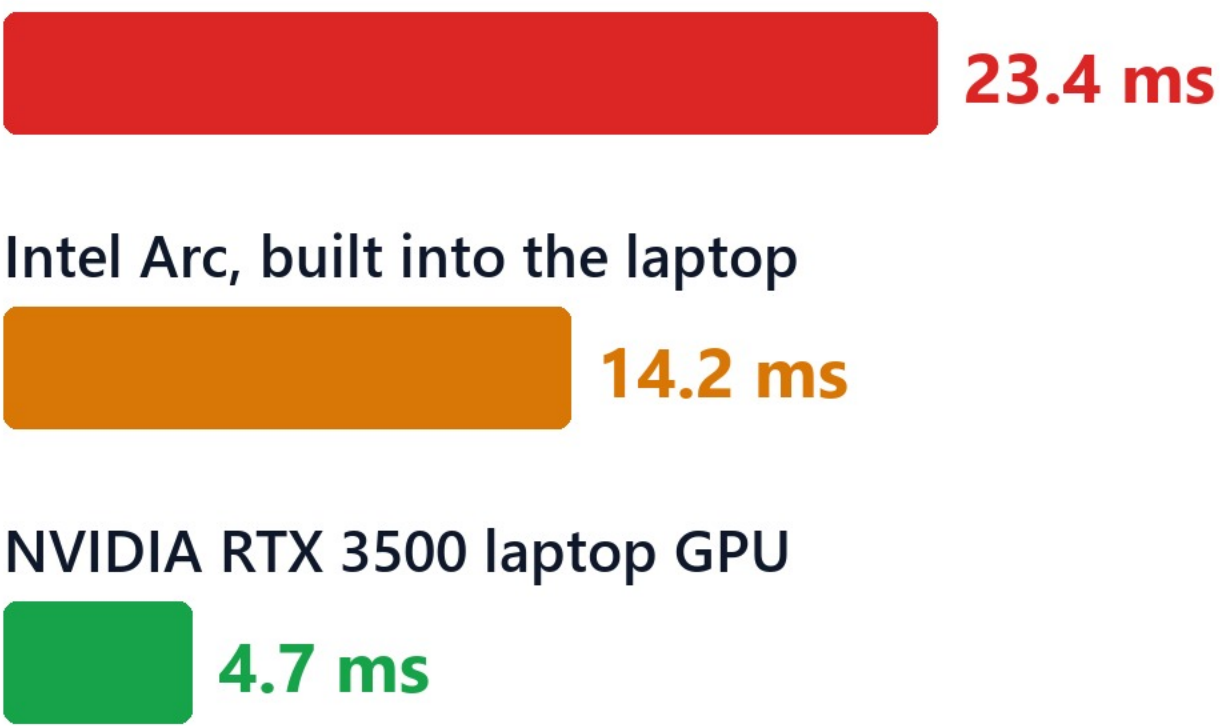
↓

CPU fallback

if the GPU can't load a model; the app shows which one it used

Same frame, three processors

YOLO11 nano finding the faces in one 1080p frame (median time).



At 4K there's a new limit: decoding a frame out of the compressed video takes 13–16 ms, longer than the GPU needs to find the faces (about 10 ms). Now the decoder sets the pace.

Trick 1: reuse one buffer

Before the model can look at a frame, the app copies it into the model's input: a grid of numbers, 640 × 640 for each color, about 5 MB.

99	0
pauses per 300 frames	pauses per 300 frames
a new buffer every frame	the same buffer, reused

- A new 4.9 MB buffer per frame kept filling the memory, so the garbage collector stopped the whole app to clean up: 99 full collections in 300 frames.
- Now one buffer lives as long as the model and is reused for every frame: no collections, and preparing a frame takes 29 % less time.

Gen 2 collections, measured over 300 frames.

Trick 2: scan 5 frames a second

At 60 frames a second, little changes from one frame to the next. So the AI looks at frames 0, 12, 24, 36 and 48 of every second.



With the large model, about 75 ms a frame:

all 60 frames: 60 × 75 ms = 4.5 s

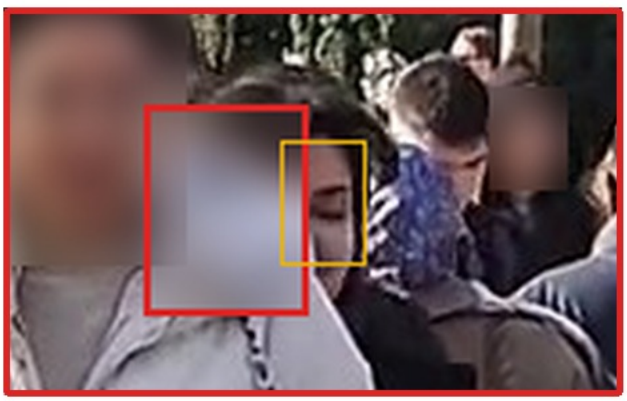
5 frames: 5 × 75 ms = 0.375 s

per second of video

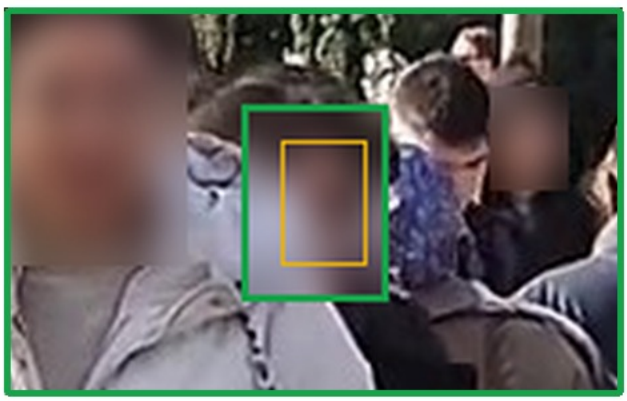
- The other 55 frames still have to be decoded (a video stores most frames as changes from the ones before), but OpenCV never turns them into color pixels.
- That alone made reading the video 2.3 times faster: 272 → 615 frames a second.
- The catch: boxes now exist only on the scanned frames.

Between scans, the box must follow the face

If a box just stays where the last scan saw it, a moving face walks out of its own blur: a privacy leak. So each box slides toward where the same face is at the next scan.



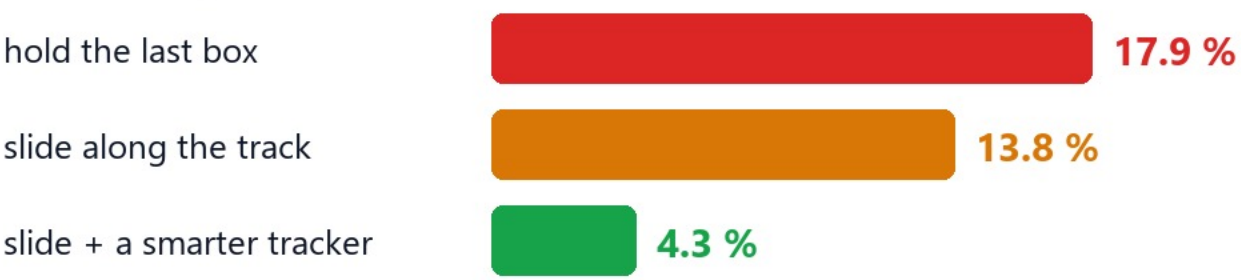
Hold the last box



Slide along the track

Yellow: the face, as the model finds it when it looks at every frame. With the held box, most of it walks out of the blur.

Boxes visibly off the face, on a one-minute test video:



Real frames of the crowd clip, frame 226, 9 frames after a scan (1080p, 60 fps).

Samit Desai · linkedin.com/in/samit-desai-62868420

Same face, same track

1. **Predict**

   where each face is heading (constant velocity)

2. **Overlap**

   join each new box to the track it overlaps most (IoU)

3. **Close and the same size**

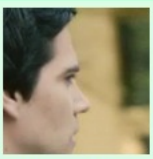
   a small face moving too fast to overlap still joins its track (within 1.5 box sizes, similar area)

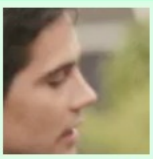
4. **Result**

   on a one-minute test video, step 3 cut the tracks from 262 to 80; with sliding, boxes visibly off the face became four times rarer (17.9 % → 4.3 %)

From tracks to people

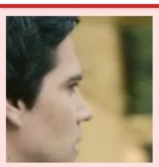
A track ends whenever someone leaves the screen: even a short film gives you over a hundred.


Thom, 4:40


Thom, 0:25

similarity 0.51

same person (≥ 0.35)


Thom, 4:40


Celia, 0:25

similarity 0.05

different people (< 0.35)

1. **3 good shots per track**
   the biggest, most confident boxes, at three different moments
2. **5 landmarks (YuNet)**
   eyes, nose and mouth corners, to line the face up
3. **512 numbers (ArcFace)**
   an embedding, like a fingerprint of the face
4. **Group**
   tracks with similar fingerprints become one suggested person
5. **One hard rule**
   two faces on the same frame are never the same person

Tears of Steel: 103 tracks → 35 suggested people

Name a person once



- I name Thom and Celia once. When Celia comes back in a new scene, she's already labeled: the app knows it's the same face.
- Grouping can miss, like a shot of Thom in profile: I merge it into Thom, and any edit can be undone.
- A face fingerprint is biometric data: grouping only runs if you allow it, and the fingerprints stay in memory, never saved.

Faces, tracks, people

- The AI finds the faces, the tracker follows them, and grouping turns them into people, all on the laptop.
- But a model can still miss a face. That's where the human comes in.

Next episode

Episode 4: the face my AI never looked at, and a fast, faithful export.

Where it's going: the whole pipeline on the GPU.

Footage: Towfiqu barbhuiya, Talha Uguz / Pexels · Tears of Steel (CC) Blender Foundation | mango.blender.org, CC BY 3.0.